UNIVERSIDADE POLITÉCNICA DO CÁVADO E DO AVE

ESCOLA SUPERIOR DE TECNOLOGIA UPCA

Sistema de Leitura para Gestão de Stock Alimentar

Nuno Miguel Fernandes Oliveira

Nº 31550 – Regime Pós-laboral

Orientação

Prof. Luís Ferreira

Ano letivo 2026/2027

Licenciatura em Engenharia de Sistemas Informáticos

Escola Superior de Tecnologia

Instituto Politécnico do Cávado e do Ave

RESUMO

A gestão de inventário de produtos alimentares exige registos fiáveis de quantidades e de datas de validade, mas a recolha destes dados é frequentemente manual, heterogénea e sujeita a erros. O presente relatório descreve o desenvolvimento de um *pipeline* ETL (*Extract, Transform, Load*) que automatiza o percurso dos dados desde a leitura de códigos de barras até ao seu armazenamento numa base de dados relacional.

A solução assenta numa separação clara de responsabilidades. O **Node-RED** atua na linha da frente, expondo quatro *endpoints* HTTP que recolhem as leituras, enriquecem os produtos através de uma API externa e geram ficheiros intermédios em CSV, JSON e XML. O **KNIME** assume o processamento pesado: validação, junção de fontes, normalização de texto com dicionários e expressões regulares, conversão de unidades através de um *script* Python e registo estruturado de erros. O **PostgreSQL** garante o armazenamento com integridade referencial. O fluxo foi testado com o **Postman**, tanto ao nível de cada *endpoint* como na integração entre o Node-RED e o KNIME. Por fim, os dados carregados são explorados em *dashboards* no **Grafana**, que permitem acompanhar a proximidade das datas de validade, a distribuição do stock por categoria e as estatísticas de execução do *pipeline*.

O *pipeline* cumpre o objetivo de transformar uma sessão de contagem num carregamento consistente da base de dados, com falhas pontuais desviadas para uma tabela de *logs* em vez de interromperem a execução. Algumas limitações foram contornadas com soluções pragmáticas e o relatório discute, para cada uma, qual seria a abordagem ideal numa arquitetura de produção.

Palavras-chave: ETL, Node-RED, KNIME, PostgreSQL, Grafana, gestão de inventário, qualidade de dados

ABSTRACT

Food inventory management requires reliable records of quantities and expiration dates, yet data collection is often manual, heterogeneous and error-prone. This report describes the development of an ETL (Extract, Transform, Load) pipeline that automates the data path from barcode scanning to storage in a relational database.

The solution is built on a clear separation of concerns. **Node-RED** acts as the front line, exposing four HTTP endpoints that capture scans, enrich products through an external API and produce intermediate CSV, JSON and XML files. **KNIME** handles the heavy processing: validation, joining of sources, text normalisation with dictionaries and regular expressions, unit conversion through a Python script and structured error logging. **PostgreSQL** provides storage with referential integrity. The flow was tested with **Postman**, both for each endpoint and for the integration between Node-RED and KNIME. Finally, the loaded data are explored in **Grafana** dashboards that track how close products are to their expiration date, the distribution of stock by category and pipeline execution statistics.

The pipeline turns a counting session into a consistent database load, with isolated failures routed to a log table instead of halting execution. Some limitations were addressed with pragmatic workarounds — running KNIME through its graphical interface instead of batch mode, assigning the CSV header in KNIME, summing quantities through a staging table and a conventional expiration date for fresh products — and the report discusses, for each one, what the ideal approach would be in a production architecture.

Keywords: ETL, Node-RED, KNIME, PostgreSQL, Grafana, inventory management, data quality

ÍNDICE

1. Introdução 1
1.1. Objetivos 1
1.2. Contexto 2
1.3. Estrutura do documento 3
2. Análise do Problema 5
2.1. Gestão de inventário alimentar 5
2.2. Desafios de qualidade de dados 5
2.3. Ferramentas selecionadas 6
3. Análise e Modelação do Sistema 9
3.1. Levantamento de requisitos 9
3.2. Arquitetura geral 9
3.3. Fluxo de dados e formatos intermédios 10
3.4. Modelo de dados 11
4. Implementação 13
4.1. Recolha e extração no Node-RED 13
4.1.1. Leitura e enriquecimento de produtos 14
4.1.2. Confirmação de itens e o cabeçalho do CSV 14
4.1.3. Produtos frescos e construção do XML 15
4.1.4. Encerramento da sessão 16
4.2. Integração entre o Node-RED e o KNIME 16
4.3. Transformação no KNIME 17
4.3.1. Extração, validação e junção 18
4.3.2. Normalização de texto 18
4.3.3. Extração e conversão de medidas 19

4.3.4. Produtos frescos e data de validade 20
4.3.5. Tratamento de erros e auditoria 20
4.4. Load no PostgreSQL 21
4.4.1. Integridade referencial 21
4.4.2. Acumulação de quantidades 22
4.5. Pós-load 23
4.6. Visualização no Grafana 23
5. Testes e Análise de Resultados 25
5.1. Estratégia de testes com o Postman 25
5.2. Síntese crítica das limitações 27
6. Conclusão 29
6.1. Trabalho futuro 29

ÍNDICE DE FIGURAS

Figura 1: Arquitetura global do pipeline ETL. 10
Figura 2: Modelo de dados no PostgreSQL. 11
Figura 3: Fluxos do Node-RED com os quatro endpoints. 13
Figura 4: Workflow do KNIME organizado por metanodes. 18
Figura 5: Extrato da tabela error_logs com registos de teste. 21
Figura 6: Dashboard do Grafana com validades, stock por categoria e estatísticas de execução (antes da normalização das categorias). 23
Figura 7: Testes aos endpoints do Node-RED no Postman. 26

ÍNDICE DE TABELAS

Tabela 1: Requisitos funcionais e não funcionais do pipeline. 9
Tabela 2: Ficheiros intermédios trocados entre o Node-RED e o KNIME. 11
Tabela 3: Endpoints expostos pelo Node-RED. 14
Tabela 4: Cenários de teste executados com o Postman. 27
Tabela 5: Síntese das dificuldades, soluções implementadas e abordagem ideal. 28

Glossário

Dashboard – Painel visual que reúne vários indicadores e gráficos sobre um conjunto de dados, atualizado a partir da fonte de dados.

Dicionário (de regras) – Tabela externa, neste projeto um ficheiro Excel, que associa códigos a nomes finais e é consumida pelo nó *Rule Engine (Dictionary)* do KNIME.

Endpoint – Ponto de acesso de um serviço HTTP, identificado por um caminho e um método, que recebe pedidos e devolve uma resposta.

Expressão regular – Padrão textual que descreve um conjunto de cadeias de carateres, usado para pesquisar, validar ou substituir texto.

Flow Variable – Variável de fluxo do KNIME que circula entre nós (ligações a vermelho) e que pode ser usada para impor a ordem de execução entre ramos independentes.

Metanode – Agrupamento de vários nós do KNIME num único bloco, usado para organizar visualmente um *workflow* por fases.

Modo batch – Execução de um *workflow* do KNIME a partir da linha de comandos, sem interface gráfica, terminando o processo no final.

Tabela intermédia (staging) – Tabela temporária onde os dados são carregados antes de serem integrados nas tabelas definitivas.

Upsert – Operação que insere um registo se este não existir ou o atualiza se já existir, tipicamente substituindo os valores anteriores.

Workaround – Solução alternativa que contorna uma limitação técnica sem a resolver na sua origem.

Siglas e Acrónimos

API – Application Programming Interface (Interface de Programação de Aplicações)

CSV – Comma-Separated Values (Valores Separados por Vírgulas)

EAN – European Article Number (Número Europeu de Artigo)

ETL – Extract, Transform, Load (Extração, Transformação e Carga)

FK – Foreign Key (Chave Estrangeira)

GUI – Graphical User Interface (Interface Gráfica de Utilizador)

HTTP – HyperText Transfer Protocol (Protocolo de Transferência de Hipertexto)

JSON – JavaScript Object Notation

KNIME – Konstanz Information Miner

PK – Primary Key (Chave Primária)

SQL – Structured Query Language (Linguagem de Consulta Estruturada)

XML – eXtensible Markup Language (Linguagem de Marcação Extensível)

1. Introdução

As organizações que lidam com produtos alimentares enfrentam um problema de dados com consequências muito concretas: um registo de inventário desatualizado ou incoerente traduz-se em ruturas de stock, em compras duplicadas e, sobretudo, em desperdício de produtos cuja validade expirou sem que ninguém o tivesse detetado. A informação necessária para evitar estas situações existe, está no código de barras de cada embalagem, nas bases de dados públicas de produtos e na data impressa no rótulo, mas encontra-se dispersa, em formatos heterogéneos e com qualidade variável.

Os processos ETL (*Extract, Transform, Load*) são a resposta clássica da engenharia de dados a este tipo de cenário: extraem dados de fontes diversas, submetem-nos a regras de limpeza e conformação, e carregam-nos num repositório consistente. O valor de um processo ETL não reside apenas na movimentação dos dados, mas na garantia de que aquilo que chega ao destino é fiável, rastreável e coerente com as regras do negócio.

Este relatório documenta o desenho e a implementação de um *pipeline* ETL para gestão de inventário alimentar, construído sobre três ferramentas complementares: o **Node-RED**, para a recolha de dados e a interação com o utilizador; o **KNIME**, para o processamento e a orquestração do *load*; o **PostgreSQL**, para o armazenamento relacional; e o **Grafana**, para a exposição dos dados em *dashboards*. Para além da descrição do que foi construído, o documento procura manter uma postura crítica, distinguindo de forma explícita as soluções efetivamente implementadas daquelas que seriam as melhores práticas numa arquitetura de produção.

1.1. Objetivos

O objetivo geral do projeto consistiu em construir um fluxo automatizado que transformasse uma sessão de contagem de inventário num carregamento consistente

de uma base de dados relacional. Este objetivo desdobra-se nos seguintes objetivos específicos:

- recolher leituras de códigos de barras através de *endpoints* HTTP e enriquecer cada produto com informação obtida de uma API externa;
- suportar o registo de produtos frescos, que não dispõem de código de barras nem, frequentemente, de data de validade explícita;
- normalizar descrições, categorias e medidas de produtos, de modo a que dados vindos de fontes diferentes sejam comparáveis;
- garantir que falhas pontuais, como um EAN inexistente na API, não interrompem o processamento do lote, ficando antes registadas para auditoria;
- carregar os dados no PostgreSQL respeitando a integridade referencial e acumulando corretamente as quantidades já existentes em stock;
- automatizar a passagem entre a fase de recolha (Node-RED) e a fase de processamento (KNIME), sem intervenção manual do operador;
- disponibilizar *dashboards* que transformem os dados carregados em informação útil para a decisão, com destaque para a proximidade das datas de validade.

1.2. Contexto

O trabalho insere-se no domínio da integração e qualidade de dados, e foi desenvolvido como projeto prático de engenharia de um *pipeline* ETL completo, desde a captura até ao *load*. O cenário de referência é o de um operador que percorre um armazém ou uma despensa, lê os códigos de barras dos produtos, confirma quantidades e datas de validade, regista manualmente os produtos frescos e, no final, encerra a sessão. A partir desse momento, todo o tratamento deve decorrer de forma automática.

A escolha das ferramentas privilegiou plataformas de programação visual e de baixo código, adequadas à prototipagem rápida e à leitura do fluxo de dados por terceiros, sem abdicar de pontos de extensão em código (funções JavaScript no Node-RED, *scripts* Python e SQL no KNIME) sempre que a lógica o exigiu.

Na elaboração deste relatório e no acompanhamento do desenvolvimento recorreu-se ao assistente de inteligência artificial Claude (Anthropic, 2026), utilizado

como apoio à redação e revisão do texto, ao esclarecimento de dúvidas técnicas e à análise de problemas encontrados durante a implementação. As decisões de implementação, a execução dos testes e a validação dos resultados são da responsabilidade do autor.

1.3. Estrutura do documento

Para além da presente introdução, o documento está organizado do seguinte modo:

1. **Análise do Problema**, que caracteriza o problema de gestão de inventário alimentar, os desafios de qualidade de dados envolvidos e as ferramentas selecionadas;
2. **Análise e Modelação do Sistema**, onde se levantam os requisitos, se apresenta a arquitetura global, o fluxo de dados entre formatos e o modelo de dados relacional;
3. **Implementação**, que descreve em detalhe os fluxos do Node-RED, a integração com o KNIME, as transformações realizada, a estratégia de load e a visualização no Grafana, incluindo as dificuldades encontradas e as soluções alternativas adotadas;
4. **Testes e Análise de Resultados**, que apresenta a estratégia de testes com o Postman e uma síntese crítica das limitações e da respetiva abordagem ideal;
5. **Conclusão**, que resume o trabalho realizado e propõe linhas de trabalho futuro.

2. Análise do Problema

Antes de desenhar qualquer solução, importa compreender a natureza do problema que se pretende resolver. Este capítulo começa por caracterizar as particularidades da gestão de inventário alimentar, com destaque para o papel da data de validade como dimensão de controlo de stock. Identifica, de seguida, os principais desafios de qualidade de dados associados às fontes utilizadas, que condicionam diretamente as transformações a implementar. Por fim, apresenta as ferramentas selecionadas e justifica a sua escolha em função das responsabilidades que cada uma assume no pipeline.

2.1. Gestão de inventário alimentar

Um inventário alimentar distingue-se de um inventário genérico por uma dimensão adicional: o tempo. Duas unidades do mesmo produto, com o mesmo EAN, não são equivalentes se tiverem datas de validade diferentes. Consequentemente, a unidade de controlo do stock não é apenas o produto, mas o par produto–validade. Esta característica condiciona de forma direta o modelo de dados e, como se verá adiante, esteve na origem de uma das decisões de modelação mais discutíveis do projeto.

Acresce que nem todos os produtos são iguais do ponto de vista da recolha. Os produtos embalados têm um código EAN que permite consultar fontes externas e obter automaticamente nome, marca, categoria e quantidade nominal. Os produtos frescos — fruta, legumes, pão — não têm código de barras normalizado e, muitas vezes, não têm uma data de validade impressa. O sistema tem, por isso, de suportar dois percursos de entrada distintos que convergem no mesmo repositório.

2.2. Desafios de qualidade de dados

Os dados obtidos de APIs públicas de produtos são, por natureza, heterogéneos. Foram identificados três tipos de problemas que o *pipeline* teria de resolver:

- **Texto não normalizado:** descrições e categorias surgem em vários idiomas, com variações ortográficas, abreviaturas e erros de escrita, o que impede agrupamentos e comparações diretas;
- **Medidas heterogéneas:** o campo de tamanho/peso aparece em formatos livres (por exemplo, com e sem espaço entre valor e unidade, ou com unidades diferentes para a mesma grandeza), enquanto a base de dados exige valores numéricos em unidades *standard*;
- **Dados em falta ou inexistentes:** EANs que a API não reconhece, produtos frescos sem data de validade e campos opcionais vazios.

Um *pipeline* robusto não pode tratar estes casos como exceções fatais. A abordagem recomendada consiste em separar o fluxo de registos válidos do fluxo de registos com erro, permitindo que o lote prossiga e que as falhas fiquem documentadas para correção posterior.

2.3. Ferramentas selecionadas

A arquitetura foi desenhada para tirar partido das valências específicas de cada ferramenta, evitando sobrecarregar qualquer uma delas com responsabilidades para as quais não está vocacionada.

O **Node-RED** é uma ferramenta de programação baseada em fluxos, construída sobre Node.js, particularmente adequada à integração de dispositivos, APIs e serviços HTTP. A sua natureza orientada a eventos torna-o indicado para a fase de recolha, em que cada leitura de código de barras é um evento independente que deve ser processado e respondido de imediato.

O **KNIME** (*Konstanz Information Miner*) é uma plataforma de análise e integração de dados baseada em *workflows* visuais compostos por nós. Oferece nós nativos para leitura de múltiplos formatos, junções, regras, tratamento de valores em falta e ligação a bases de dados, bem como a possibilidade de integrar *scripts* Python. É, por isso, a ferramenta indicada para o processamento pesado e para a orquestração transacional do *load*.

O **PostgreSQL** é um sistema de gestão de bases de dados relacional de código aberto, escolhido pela robustez no suporte a restrições de integridade (chaves primárias, chaves estrangeiras, `NOT NULL`) e pela expressividade do seu dialeto SQL, nomeadamente a cláusula `ON CONFLICT` para operações de inserção condicional.

O **Grafana** é uma plataforma de observabilidade e visualização que se liga diretamente a fontes de dados como o PostgreSQL e permite construir *dashboards* a partir de consultas SQL, com tabelas, gráficos e regras de cor por limiares. Foi usado como camada de apresentação, fechando o ciclo do *pipeline*: os dados não são apenas carregados, mas tornados acessíveis a quem gere o inventário.

Por fim, o **Postman** foi utilizado como ferramenta de teste. Permite construir e enviar pedidos HTTP com corpos JSON arbitrários, inspecionar as respostas e reproduzir sequências de pedidos de forma controlada. Desempenhou o papel do leitor de códigos de barras durante o desenvolvimento, simulando sessões de contagem completas.

A instalação e a configuração das ferramentas seguiram a documentação oficial de cada uma (Grafana Labs, s.d.; KNIME AG, s.d.; OpenJS Foundation, s.d.; Postman, Inc., s.d.; The PostgreSQL Global Development Group, s.d.). No caso do Node-RED, a documentação foi também consultada para identificar os requisitos e os comandos de instalação através do terminal.

3. Análise e Modelação do Sistema

Este capítulo traduz o problema descrito anteriormente numa especificação do sistema a construir. São primeiro levantados os requisitos funcionais e não funcionais que o *pipeline* deve cumprir. Com base neles, apresenta-se a arquitetura global da solução, organizada pelas fases do processo ETL, e descreve-se o fluxo de dados entre o Node-RED e o KNIME, incluindo a justificação dos formatos intermédios adotados. O capítulo termina com o modelo de dados relacional implementado no PostgreSQL, cujas decisões de modelação têm impacto direto em várias das soluções discutidas no capítulo de implementação.

3.1. Levantamento de requisitos

A partir dos objetivos definidos foram identificados os requisitos funcionais e não funcionais que orientaram o desenho da solução, sintetizados na Tabela 1.

ID	Requisito
RF01	Receber a leitura de um código de barras e devolver os dados do produto obtidos de uma API externa.
RF02	Registar a confirmação de um item lido, com quantidade e data de validade, em ficheiros persistentes.
RF03	Registar produtos frescos sem código de barras.
RF04	Encerrar a sessão de contagem e despoletar automaticamente o processamento no KNIME.
RF05	Normalizar texto (descrições e categorias) e converter medidas para unidades *standard*.
RF06	Registar em base de dados os registos que falham, indicando a fase e o motivo da falha.
RF07	Somar as quantidades lidas ao stock existente, em vez de as substituir.
RF08	Disponibilizar *dashboards* com as validades, o stock por categoria e as estatísticas de execução.
RNF01	Robustez: uma falha pontual não deve interromper o processamento do lote.
RNF02	Integridade: o *load* deve respeitar as chaves primárias e estrangeiras do modelo.
RNF03	Rastreabilidade: cada erro deve ser auditável a posteriori.
RNF04	Automação: após o fecho da sessão, o processo não deve exigir intervenção manual.

Tabela 1: Requisitos funcionais e não funcionais do pipeline.

3.2. Arquitetura geral

A Figura 1 apresenta a arquitetura global do sistema, organizada nas fases clássicas de um processo ETL. A decisão estruturante foi a **separação de responsabilidades** entre o Node-RED e o KNIME: o primeiro gere a interação, a

recolha dos códigos de barras, a integração com a API de enriquecimento e a formatação inicial dos dados; o segundo assume as transformações de texto complexas, a limpeza de dados e a gestão das transações com o PostgreSQL.

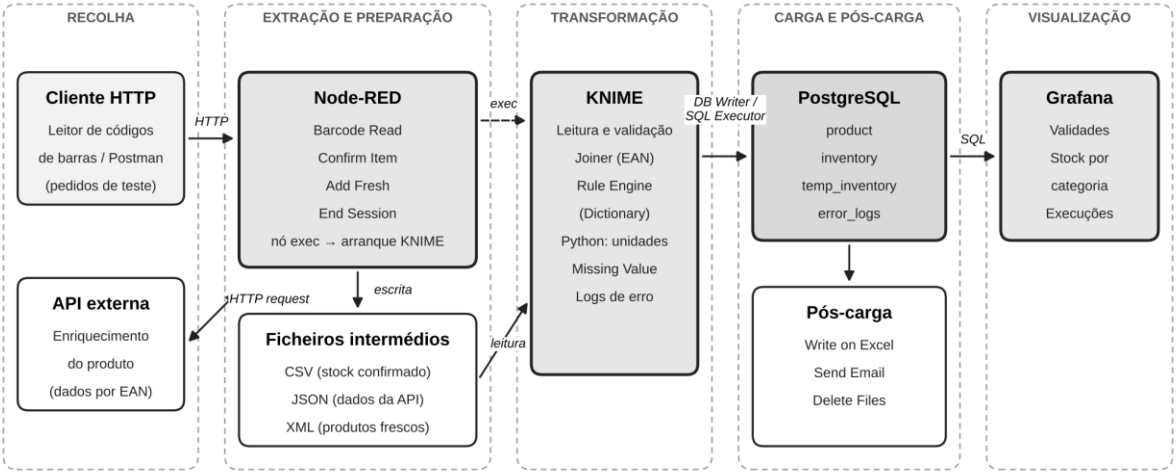


Figura 1: Arquitetura global do pipeline ETL.

A comunicação entre as duas plataformas faz-se por dois canais. O primeiro é um canal de dados assíncrono, materializado em ficheiros intermédios escritos pelo Node-RED e lidos pelo KNIME. O segundo é um canal de controlo: no momento do encerramento da sessão de contagem, o Node-RED despoleta a execução do *workflow* do KNIME através de um nó `exec`, o que garante a continuidade do processo sem intervenção do operador. A jusante do *load*, o Grafana consulta diretamente o PostgreSQL, constituindo uma camada de visualização desacoplada do processamento.

3.3. Fluxo de dados e formatos intermédios

Cada formato intermédio foi escolhido em função da natureza dos dados que transporta e do momento em que é produzido. A Tabela 2 resume esta correspondência.

Ficheiro	Produzido por	Conteúdo	Consumido no KNIME por
CSV	Confirm Item	Registo de stock confirmado (EAN, quantidade, validade), acrescentado linha a linha.	CSV Validation

Ficheiro	Produzido por	Conteúdo	Consumido no KNIME por
JSON	Confirm Item	Dados de enriquecimento do produto devolvidos pela API.	API Extraction
XML	Add Fresh / End Session	Produtos frescos acumulados durante a sessão, encapsulados no fecho.	XML Reader

Tabela 2: Ficheiros intermédios trocados entre o Node-RED e o KNIME.

O formato CSV adequa-se a um registo de *append* sequencial, linha a linha. O JSON preserva a estrutura hierárquica das respostas da API. O XML, por seu lado, exige um documento bem formado com um elemento raiz, o que obrigou a uma estratégia de acumulação descrita na secção 4.1.3.

3.4. Modelo de dados

O modelo relacional no PostgreSQL é composto por cinco tabelas, representadas na Figura 2, que se dividem em três grupos: o catálogo e o stock (`products` e `inventory`), a área de preparação do *load* (`temp_inventory`) e a auditoria (`error_logs` e `pipeline_executions`).

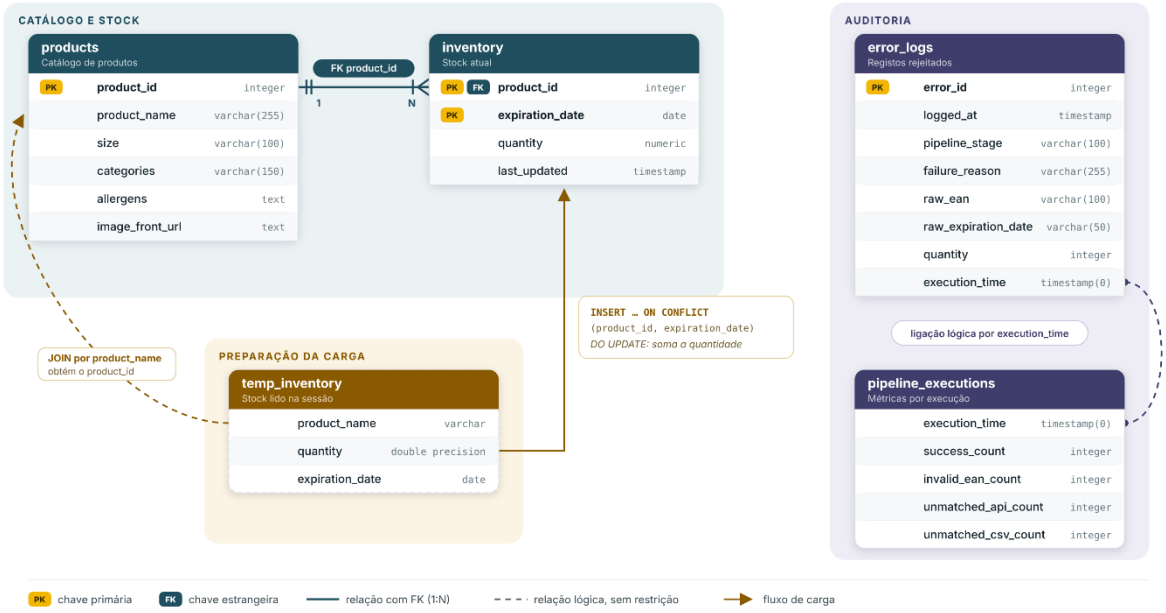


Figura 2: Modelo de dados no PostgreSQL.

- `products` — catálogo de produtos, identificados por uma chave substituta `product_id`, com os atributos já normalizados: `product_name`, `size`, `categories`, `allergens` e `image_front_url`;
- `inventory` — stock atual, com chave composta por `product_id` (chave estrangeira para `products`) e `expiration_date`, a quantidade (`quantity`) e a data da última atualização (`last_updated`);
- `temp_inventory` — tabela intermédia que recebe o stock lido na sessão (`product_name`, `quantity`, `expiration_date`) antes da sua integração em `inventory`;
- `error_logs` — registo de auditoria dos registos rejeitados, com identificador próprio, momento do registo, fase (`pipeline_stage`), motivo (`failure_reason`), os valores em bruto (`raw_ean`, `raw_expiration_date`), a quantidade e o momento da execução (`execution_time`);
- `pipeline_executions` — uma linha por execução do *pipeline*, com os contadores de registos carregados com sucesso, de EANs inválidos e de registos sem correspondência na API ou no CSV.

A chave de `inventory` depende da data de validade, coerentemente com a observação de que a unidade de controlo é o par produto–validade; é esta restrição de unicidade sobre (`product_id`, `expiration_date`) que suporta a cláusula `ON CONFLICT` usada no *load*. A coluna `expiration_date` foi definida como `NOT NULL`. Esta escolha, aparentemente inócua, teve consequências relevantes no tratamento dos produtos frescos, analisadas na secção 4.3.4.

Duas outras decisões merecem nota. Em `error_logs`, o EAN e a data de validade são guardados como texto (`raw_ean`, `raw_expiration_date`), e não com os tipos finais: um registo rejeitado pode ter precisamente um valor mal formado, e a auditoria deve preservá-lo tal como foi recebido. Por outro lado, a tabela `products` não guarda o EAN, e a ligação entre `temp_inventory` e `products` faz-se pelo nome do produto. Esta opção permite tratar da mesma forma os produtos embalados e os produtos frescos, que não têm EAN.

4. Implementação

Este capítulo descreve a implementação de cada componente, seguindo o percurso dos dados. As dificuldades encontradas são apresentadas no ponto do fluxo em que surgiram, juntamente com a solução adotada e, quando aplicável, a solução que seria preferível numa arquitetura ideal.

4.1. Recolha e extração no Node-RED

O fluxo do Node-RED (Figura 3) é composto por quatro cadeias independentes, cada uma iniciada por um nó `http in` que expõe um *endpoint* e terminada por um nó `http response` que devolve o resultado ao cliente. Todos os *endpoints* aceitam pedidos `POST` com corpo JSON e estão disponíveis na porta padrão do Node-RED (`http://localhost:1880`).

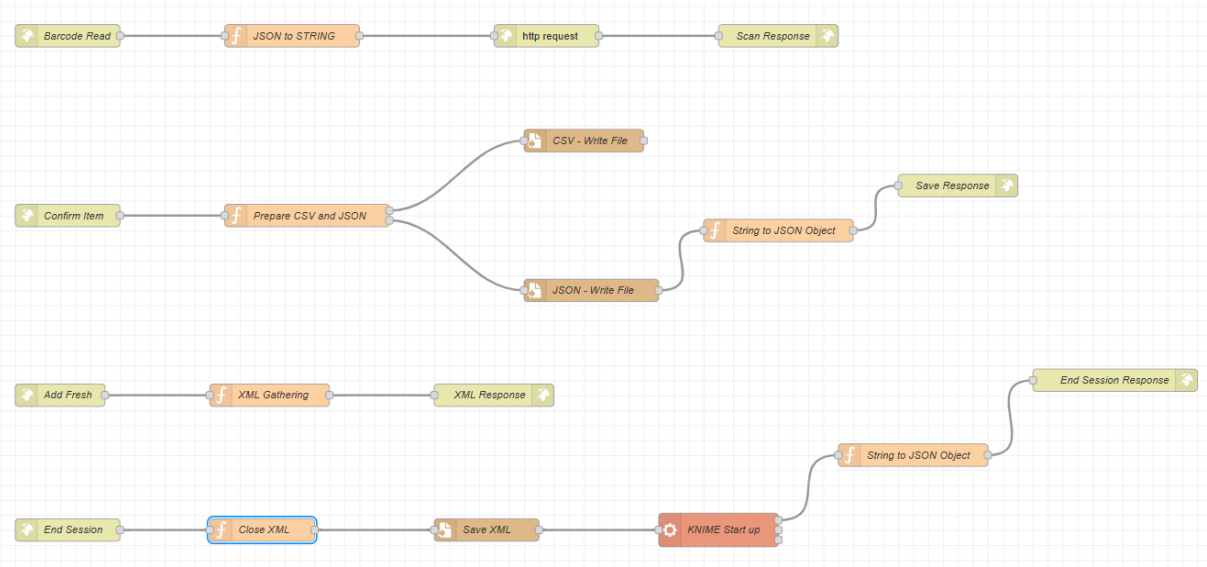


Figura 3: Fluxos do Node-RED com os quatro endpoints.

A Tabela 3 descreve a função de cada *endpoint* e a respetiva cadeia de nós.

Endpoint	Cadeia de nós	Função
Barcode Read (POST `/scan`)	JSON to STRING → http request → Scan Response	Recebe o EAN lido, consulta a API externa e devolve os dados do produto ao cliente.
Confirm Item (POST `/confirm-item`)	Prepare CSV and JSON → CSV - Write File / JSON - Write File → String to JSON Object → Save Response	Confirma o item com quantidade e validade; escreve a linha no CSV e os dados de enriquecimento no JSON.
Add Fresh (POST `/add-fresh`)	XML Gathering → XML Response	Regista um produto fresco e acumula-o em memória.
End Session (POST `/end-session`)	Close XML → Save XML → KNIME Start up → String to JSON Object → End Session Response	Encapsula e grava o XML e despoleta o workflow do KNIME.

Tabela 3: Endpoints expostos pelo Node-RED.

4.1.1. Leitura e enriquecimento de produtos

O *endpoint* **Barcode Read** (`/scan`) recebe um corpo JSON com o código lido (por exemplo, `{"ean": "5601049132995"}`) e, através de um nó `function` (*JSON to STRING*), prepara o pedido para a API de enriquecimento. O nó `http request` realiza a chamada e a resposta é imediatamente devolvida ao cliente pelo nó *Scan Response*. A resposta é o objeto JSON da API, com o código e o bloco `product`. Esta separação permite ao operador visualizar os dados do produto antes de confirmar a leitura, o que constitui uma primeira barreira contra leituras erradas.

4.1.2. Confirmação de itens e o cabeçalho do CSV

O *endpoint* **Confirm Item** (`/confirm-item`) recebe o item validado pelo operador, com os campos `ean`, `quantity` e `expiration_date` e, no campo `api_data`, os dados de enriquecimento devolvidos anteriormente pelo `/scan`. O cliente reenvia, assim, a resposta da API, o que evita uma segunda consulta e garante que o que é gravado corresponde exatamente ao que o operador viu. O nó *Prepare CSV and JSON* tem duas saídas: a primeira alimenta o nó *CSV - Write File*, que acrescenta uma linha ao ficheiro de stock; a segunda alimenta o nó *JSON - Write File*, que grava os dados de enriquecimento. Como o nó de escrita só aceita texto, o conteúdo segue nesse formato até ao fim do fluxo. Para que o cliente receba um objeto JSON, e não texto, foi acrescentado um nó `function` (*String to JSON Object*) que volta a converter o conteúdo e o envolve numa resposta normalizada, com os campos `status`, `message` e `data`, antes do nó *Save Response*.

Surgiu aqui uma dificuldade não ultrapassada. Como o ficheiro CSV é fisicamente apagado no início de cada nova sessão de inventário, o primeiro *append* deveria incluir o cabeçalho. A forma natural de o fazer seria verificar, dentro do nó `function`, se o ficheiro existe em disco. Contudo, a tentativa de utilizar o módulo nativo `fs` do Node.js resultou no erro crítico `ReferenceError: require is not defined`, uma vez que, por razões de segurança, os nós `function` do Node-RED não têm acesso direto à função `require`.

A solução alternativa adotada consistiu em gerar o CSV apenas com os dados brutos, sem cabeçalho, transferindo a resolução para o KNIME: o nó de leitura do CSV foi configurado para não esperar cabeçalho e, de seguida, um nó nativo de renomeação de colunas (`Column Renamer`) atribui os nomes corretos. Do ponto de vista crítico, esta abordagem funciona mas cria um acoplamento implícito: a ordem das colunas passa a ser um contrato não documentado entre as duas ferramentas. Idealmente, o módulo `fs` seria disponibilizado ao nó através da configuração de módulos do próprio Node-RED (via `functionGlobalContext` ou do separador de módulos do nó `function`), ou o cabeçalho seria escrito pelo fluxo de abertura de sessão, imediatamente após a limpeza do ficheiro.

4.1.3. Produtos frescos e construção do XML

O *endpoint* **Add Fresh** (`/add-fresh`) regista produtos frescos, recebendo apenas `product_name`, `category` e `quantity` e respondendo com `{"status": "OK"}`. A ausência de EAN e de data de validade neste corpo é intencional, e explica o tratamento descrito na secção 4.3.4. O desafio residiu na natureza do formato XML: ao contrário do CSV, não é possível acrescentar linhas a um ficheiro XML e manter um documento bem formado, porque todos os elementos têm de estar contidos num único elemento raiz.

A solução foi uma **arquitetura de acumulação**. O nó *XML Gathering* guarda cada item em memória (no contexto do fluxo) à medida que é registado. Apenas no evento de fecho da sessão o nó *Close XML* encapsula a coleção acumulada com as *tags* de abertura e fecho do elemento raiz, e o nó *Save XML* grava o documento completo em disco. O excerto seguinte ilustra a lógica de acumulação (código simplificado):

```
// XML Gathering — acumula o item no contexto do fluxo
let itens = flow.get('frescos') || [];
itens.push(msg.payload);
flow.set('frescos', itens);

// Close XML — encapsula a coleção no fecho da sessão
let itens = flow.get('frescos') || [];
msg.payload = '<products>'
    + itens.map(toXml).join('') + '</products>';
flow.set('frescos', []);
```

Esta opção tem a vantagem de garantir que o KNIME lê sempre um documento válido. A desvantagem é que os dados residem em memória volátil até ao fecho da sessão: uma reinicialização do Node-RED a meio de uma contagem implicaria a perda dos produtos frescos registados. Numa versão de produção, seria recomendável configurar um armazenamento de contexto persistente.

4.1.4. Encerramento da sessão

O *endpoint* **End Session** (`/end-session`) fecha e grava o XML e, através do nó *KNIME Start up* (um nó `exec`), lança a execução do *workflow* do KNIME. O pedido não precisa de corpo: é o próprio caminho que identifica a ação, pelo que basta um `POST` vazio. A saída do nó `exec` é texto (aquilo que o processo escreve no terminal), pelo que um nó `function` (*String to JSON Object*) a converte numa resposta JSON com os campos `status`, `message` e `output`, este último com a saída original do comando. Só depois é devolvida a resposta ao cliente. Este encadeamento garante que o processamento começa apenas quando todos os ficheiros intermédios estão completos.

4.2. Integração entre o Node-RED e o KNIME

A intenção original era executar o KNIME em **modo batch**, isto é, invocá-lo a partir da linha de comandos para correr o *workflow* em segundo plano, de forma invisível, terminando o processo no final. Esta é a abordagem correta para uma integração automatizada, por ser determinística, não depender de uma sessão gráfica

e devolver um código de saída que permite ao Node-RED saber se a execução teve sucesso.

Contudo, a chamada por linha de comandos gerou erros que impediram o sucesso desta execução silenciosa, e não foi possível resolvê-los no âmbito do projeto. Optou-se, por isso, por uma integração via **interface gráfica**: o nó `exec` foi mantido, mas configurado para abrir visualmente o KNIME. Para automatizar o processo sem recorrer ao modo *batch*, recorreu-se à simulação de atalhos de teclado, de modo a que, assim que a janela do KNIME abre, o *workflow* sofra o *reset* de todos os nós e seja executado na íntegra.

Esta solução cumpre o requisito de automação (RNF04) num ambiente de demonstração, mas é frágil: depende de existir uma sessão gráfica ativa, do tempo de arranque da aplicação, do foco da janela e da configuração do teclado, e não devolve ao Node-RED qualquer indicação sobre o sucesso ou o insucesso da execução. Numa arquitetura ideal, o diagnóstico do modo *batch* seria retomado, ou a execução seria delegada num servidor KNIME com interface REST, permitindo uma chamada HTTP com resposta estruturada.

4.3. Transformação no KNIME

O *workflow* do KNIME (Figura 4) está organizado em metanodes que correspondem a fases lógicas do processamento: extração (*XML Reader*, *API Extraction*, *CSV Validation*), junção (*Joiner*), processamento do stock válido (*Valid Stock Processing*), consolidação de erros (*Logs Consolidation*), preparação da auditoria (*Audit Preparation*), *load* (*Load Data*) e pós-*load* (*Write on Excel*, *Delete Files*, *Send Email*).

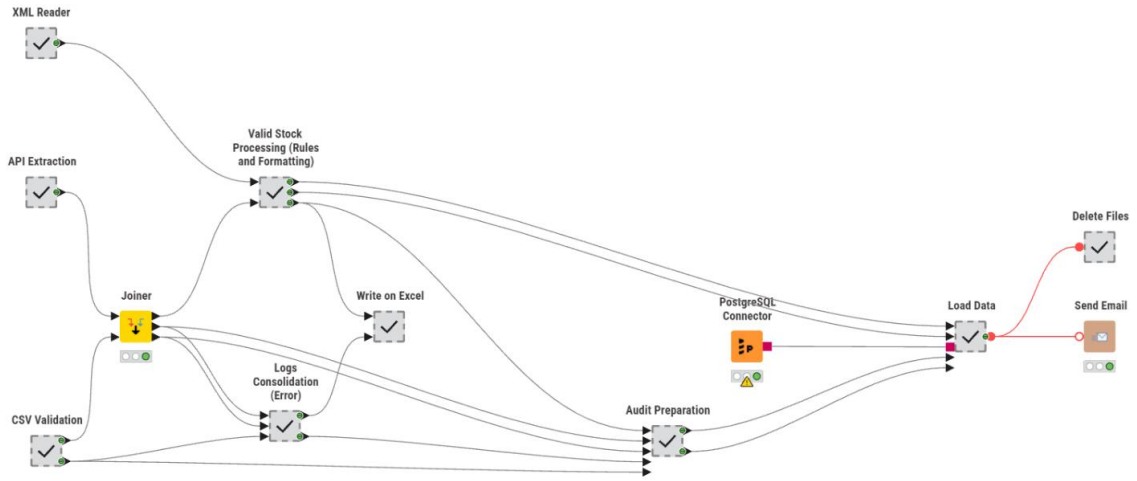


Figura 4: Workflow do KNIME organizado por metanodes.

4.3.1. Extração, validação e junção

Os três ficheiros intermédios são lidos por metanodes dedicados. O stock confirmado (CSV) passa pelo metanode *CSV Validation*, onde, entre outros passos, são atribuídos os nomes às colunas. Os dados de enriquecimento (JSON) são extraídos no metanode *API Extraction*. O nó **Joiner** associa as duas fontes pelo EAN: os registos com correspondência seguem para o processamento do stock válido, enquanto os registos sem correspondência — por exemplo, EANs que a API não reconheceu — são encaminhados para o fluxo de erros. Antes da junção, o metanode *CSV Validation* verifica também o formato dos códigos, rejeitando os que não cumprem o padrão EAN-13. Os produtos frescos, lidos pelo *XML Reader*, entram diretamente na fase de processamento, uma vez que não dependem do enriquecimento por EAN.

4.3.2. Normalização de texto

Na primeira versão do *pipeline*, a limpeza e a substituição de termos foram implementadas num único ficheiro Excel externo, consumido através do nó `Rule Engine (Dictionary)`, em que cada linha continha, simultaneamente, o padrão a reconhecer (uma expressão regular sobre o `product_name`) e o termo final. As categorias, por sua vez, eram carregadas tal como vinham da API. Esta solução misturava duas responsabilidades distintas e deixava as categorias por normalizar, o que se tornou evidente no *dashboard* do Grafana (secção 4.6).

A normalização foi, por isso, reestruturada em duas etapas, aplicadas tanto às categorias como aos nomes dos produtos. Os novos nós foram colocados no metanode *Valid Stock Processing*, logo após o nó `Concatenate` que junta os produtos embalados e os frescos, para que todas as saídas do metanode recebam dados já normalizados:

1. **Deteção (regex → código):** dois nós `Rule Engine` nativos, um para `categories` e outro para `product_name`, aplicam expressões regulares com palavras-chave em português, espanhol, inglês e francês e devolvem um código neutro (por exemplo, `PADARIA`, `LEITE` ou `PEIXE_MARISCO`);
2. **Tradução (código → nome):** dois nós `Rule Engine (Dictionary)`, alimentados pelos ficheiros `categories_dictionary.xlsx` e `products_dictionary.xlsx`, limitam-se a converter cada código no nome final, numa correspondência direta e sem expressões regulares.

As categorias convergem para uma lista fechada de 17 valores: Congelados, Enlatados e conservas, Bebidas, Iogurtes, Queijos, Leite, Outros laticínios, Ovos, Carne e charcutaria, Peixe e marisco, Padaria, Massas, arroz e cereais, Molhos, óleos e condimentos, Doces e snacks, Fruta, Legumes e Outros, sendo esta última atribuída a categorias vazias ou sem correspondência. Como o `Rule Engine` aplica a primeira regra satisfeita, a ordem das regras é relevante: "Gelados de chocolate" é classificado como Congelados, e não como Doces e snacks, porque a regra dos congelados é avaliada primeiro. Nos nomes dos produtos, quando nenhuma regra é satisfeita, mantém-se o nome original. As regras foram validadas com um conjunto de exemplos representativos antes de serem integradas no *workflow*.

Esta separação torna o dicionário mais fácil de manter por pessoas sem conhecimentos de expressões regulares, reduz o número de entradas (uma por termo canónico, em vez de uma por variante) e coloca a lógica técnica sob o controlo de versões do *workflow*. Para mudar o nome de uma categoria, ou passar os nomes para outra língua, basta editar o Excel, sem alterar as regras.

4.3.3. Extração e conversão de medidas

O tratamento do campo de tamanho/peso foi dividido em duas fases. Na primeira, a normalização dos valores brutos foi efetuada através de expressões regulares em nós nativos do KNIME, uniformizando a forma como o valor e a unidade são escritos. Na segunda, a conversão para as unidades *standard* exigidas pela base

de dados foi realizada por um *script* Python integrado no *workflow*. Este *script* recorre igualmente a expressões regulares para detetar a unidade de origem e aplica a lógica matemática de conversão correspondente.

A divisão revelou-se acertada: os nós nativos são adequados a substituições textuais, mas a lógica condicional de conversão, dependente da unidade detetada, é expressa de forma mais clara e testável em código.

4.3.4. Produtos frescos e data de validade

Os produtos frescos não possuem, em muitos casos, uma data de validade explícita. Como a base de dados impõe a restrição `NOT NULL` sobre `expiration_date`, e como esta coluna integra a chave do inventário, foi necessário recorrer a um estratagema: o nó `Missing Value` do KNIME foi configurado para inserir a data convencional **2099-12-31** em todos os registos sem validade.

Esta não é a abordagem ideal, e importa explicitá-lo. Uma data fictícia é um valor *sentinela* que contamina os dados: qualquer consulta que ordene produtos por proximidade da validade, ou que calcule prazos médios, tem de conhecer e excluir esta convenção. O problema tem origem na modelação e deveria ter sido resolvido nesse plano, por exemplo, criando uma chave primária que não dependesse da data de validade (um identificador próprio do lote), o que permitiria que `expiration_date` aceitasse valores nulos e que os produtos frescos fossem tratados de forma mais flexível.

4.3.5. Tratamento de erros e auditoria

Para evitar que falhas pontuais quebrassem a execução de todo o *pipeline*, foi implementado um fluxo de desvio. Os registos com falhas, provenientes de diferentes fases (validação do CSV, junção com a API, regras de processamento), são reunidos no metanode *Logs Consolidation (Error)*, preparados no metanode *Audit Preparation* e inseridos na tabela `error_logs`. Cada registo indica a fase do *pipeline* em que ocorreu a falha (`pipeline_stage`), a razão da falha (`failure_reason`), os valores em bruto do EAN e da data de validade (`raw_ean`, `raw_expiration_date`), a quantidade e o momento da execução. Em paralelo, o metanode *Audit Preparation* calcula os contadores de cada execução e grava-os em `pipeline_executions`.

A Figura 5 mostra um extrato real da tabela `error_logs` obtido durante os testes. Surgem os dois tipos de falha mais frequentes: na fase *EAN Validation*, códigos que não cumprem o formato EAN-13 (`EAN-13 format`), como `56010491329`, com apenas 11

dígitos; e na fase *API Enrichment (Join)*, códigos válidos que a API não reconhece (`EAN not found in API`), como `2000000000010`. A repetição dos mesmos registos em execuções sucessivas corresponde às várias corridas de teste com os mesmos dados de entrada, e mostra que cada execução deixa o seu próprio rasto auditável.

	error_id [PK] integer	logged_at timestamp without time zone	pipeline_stage character varying (100)	failure_reason character varying (255)	raw_ean character varying (100)	raw_expiration_date character varying (50)	quantity integer	execution_time timestamp without time zone
1	1	2026-10-15 04:03:15.151639	EAN Validation	EAN-13 format	56010491329	27/03/2027	70	2026-10-15 04:03:15
2	2	2026-10-15 04:05:18.307418	EAN Validation	EAN-13 format	56010491329	27/03/2027	70	2026-10-15 04:05:18
3	3	2026-10-15 04:08:49.876915	EAN Validation	EAN-13 format	56010491329	27/03/2027	70	2026-10-15 04:08:50
4	4	2026-10-15 04:10:54.35136	API Enrichment (Join)	EAN not found in API	2000000000010	05/01/2028	1	2026-10-15 04:10:54
5	5	2026-10-15 04:10:54.35136	EAN Validation	EAN-13 format	56010491329	27/03/2027	70	2026-10-15 04:10:54
6	6	2026-10-15 04:26:28.449691	API Enrichment (Join)	EAN not found in API	2000000000010	05/01/2028	1	2026-10-15 04:26:28
7	7	2026-10-15 04:26:28.449691	EAN Validation	EAN-13 format	56010491329	27/03/2027	70	2026-10-15 04:26:28
8	8	2026-10-15 04:27:39.639404	API Enrichment (Join)	EAN not found in API	2000000000010	05/01/2028	1	2026-10-15 04:27:39

Figura 5: Extrato da tabela error_logs com registos de teste.

Durante a implementação deste fluxo ocorreu um erro de execução (*batch entry aborted*) na escrita para o PostgreSQL. A causa era um desalinhamento de esquemas: o KNIME tentava mapear a coluna `expiration_date`, que a tabela `error_logs` não possuía (na versão final, a data é guardada em bruto na coluna `raw_expiration_date`). A resolução exigiu uma validação rigorosa entre o *output* do KNIME e o esquema da base de dados, garantindo que todas as colunas de rastreio de erros estavam alinhadas. A lição retirada é que o esquema da base de dados funciona como um contrato, e que alterações num dos lados devem ser verificadas sistematicamente no outro.

4.4. *Load* no PostgreSQL

O *load* é realizado no metanode *Load Data*, que recebe a ligação à base de dados a partir do nó **PostgreSQL Connector**, bem como os fluxos de stock válido e de auditoria.

4.4.1. Integridade referencial

A chave estrangeira de `inventory` para `products` obriga a que os produtos sejam inseridos antes do stock que os referencia. No KNIME, ramos sem dependência de dados entre si podem ser executados por qualquer ordem, pelo que foi necessário impor essa ordem explicitamente. Utilizaram-se as ligações de variáveis de fluxo (*Flow Variables*, representadas a vermelho) para garantir que o nó `DB Writer` da tabela `products` termina a inserção antes de o fluxo tentar associar esses produtos na tabela `inventory`.

4.4.2. Acumulação de quantidades

A intenção original era utilizar o nó `DB Merger` para somar a quantidade lida à quantidade já existente em `inventory`. Esta abordagem revelou-se impossível: o nó realiza um *upsert* padrão, substituindo integralmente o valor existente em vez de o incrementar, e não permite operações aritméticas durante a execução. Usá-lo levaria a que cada nova contagem apagasse silenciosamente o stock anterior.

Para contornar a limitação, o *load* foi reorganizado em dois passos. Primeiro, um nó `DB Insert` grava todo o stock lido na sessão na tabela intermédia `temp_inventory`. Depois, um nó `DB SQL Executor` executa uma consulta SQL que gere os conflitos: insere os registos novos em `inventory` ou, caso o par produto–validade já exista, soma a quantidade da tabela temporária ao stock existente. A consulta utilizada é a seguinte:

```
INSERT INTO inventory
  (product_id, expiration_date, quantity)
SELECT p.product_id, t.expiration_date, t.quantity
FROM temp_inventory t
JOIN products p ON t.product_name = p.product_name
ON CONFLICT (product_id, expiration_date)
DO UPDATE
  SET quantity = inventory.quantity + EXCLUDED.quantity;
```

Embora tenha surgido como alternativa, este padrão de tabela intermédia seguida de integração em SQL corresponde, na verdade, a uma boa prática de engenharia de dados: torna o *load* auditável, permite validar o lote antes da integração e delega na base de dados uma operação que esta executa de forma atómica e eficiente.

A consulta tem, contudo, uma fragilidade que convém explicitar. A ausência de agregação: se a mesma combinação de produto e validade aparecer duas vezes na mesma sessão, o PostgreSQL rejeita a instrução, porque um `ON CONFLICT DO UPDATE` não pode afetar a mesma linha duas vezes no mesmo comando.

4.5. Pós-*load*

Após o *load*, o *workflow* executa três ações complementares: o metanode *Write on Excel* exporta os resultados do processamento e os erros consolidados para um ficheiro Excel de consulta; o metanode *Delete Files* remove os ficheiros intermédios, preparando o sistema para a sessão seguinte; e o nó *Send Email* envia uma notificação de conclusão. A remoção dos ficheiros e o envio do *email* estão condicionados ao fim do *load* através de variáveis de fluxo, evitando que os dados de origem sejam apagados antes de estarem persistidos.

4.6. Visualização no Grafana

A última camada do sistema é um *dashboard* no **Grafana**, ligado diretamente ao PostgreSQL, que traduz os dados carregados em informação de gestão (Figura 6). Foram construídos três painéis, escolhidos em função das questões mais relevantes para um inventário alimentar.

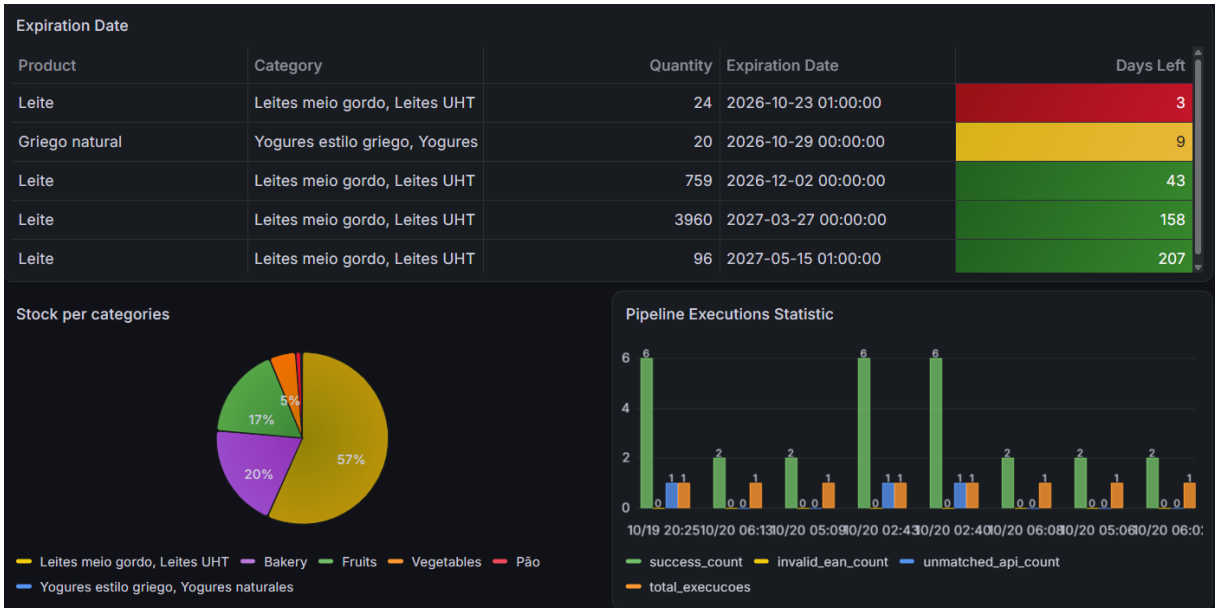


Figura 6: Dashboard do Grafana com validades, stock por categoria e estatísticas de execução (antes da normalização das categorias).

- **Expiration Date** — tabela com produto, categoria, quantidade, data de validade e dias restantes. A coluna *Days Left* usa um código de cores por limiares (vermelho para validades iminentes, amarelo para validades próximas e verde para as restantes), respondendo diretamente ao problema do desperdício identificado no capítulo 2;

- **Stock per categories** — gráfico circular com a distribuição do stock por categoria, que permite identificar rapidamente o peso relativo de cada família de produtos;
- **Pipeline Executions Statistic** — gráfico de barras que, para cada execução do *pipeline*, apresenta o número de registos carregados com sucesso (`success_count`), de EANs inválidos (`invalid_ean_count`), de produtos sem correspondência na API (`unmatched_api_count`) e o total (`total_execucoes`), calculados a partir da tabela `pipeline_executions`. Este painel dá visibilidade operacional ao tratamento de erros: uma execução com falhas não passa despercebida.

O *dashboard* teve também um valor de diagnóstico inesperado. Na versão inicial do *pipeline*, o painel de categorias (Figura 6) mostrou que a normalização de texto não era completa: coexistiam categorias compostas ("Leites meio gordo, Leites UHT"), categorias em espanhol ("Yogures estilo griego, Yogures naturales"), em inglês ("Bakery", "Fruits", "Vegetables") e em português ("Pão"), sendo "Bakery" e "Pão" semanticamente equivalentes. Esta evidência motivou a reestruturação descrita na secção 4.3.2, com a separação entre deteção e tradução e a adoção de uma lista fechada de categorias.

5. Testes e Análise de Resultados

A validação de um *pipeline* ETL não se esgota na verificação de que cada componente funciona isoladamente: é na interação entre ferramentas, formatos e esquemas que surgem muitos dos problemas. Este capítulo descreve a estratégia de testes seguida, baseada no Postman, que abrangeu tanto os *endpoints* do Node-RED como a integração de ponta a ponta com o KNIME e o PostgreSQL. Apresenta, depois, uma síntese crítica das limitações encontradas ao longo do projeto, confrontando as soluções efetivamente implementadas com a abordagem que seria ideal numa arquitetura de produção.

5.1. Estratégia de testes com o Postman

Os testes foram realizados com o **Postman**, que substituiu o leitor de códigos de barras durante o desenvolvimento e permitiu controlar com precisão o conteúdo de cada pedido. A estratégia foi organizada em dois níveis.

No primeiro nível, cada um dos **quatro endpoints** do Node-RED foi testado isoladamente, verificando o código de estado HTTP, o conteúdo da resposta e os efeitos secundários esperados (a consulta à API, a escrita dos ficheiros ou a acumulação em memória). No segundo nível, foi testada a **integração entre o Node-RED e o KNIME**, simulando uma sessão completa: uma sequência de leituras e confirmações, o registo de produtos frescos e o pedido de encerramento da sessão, seguido da verificação de que o KNIME arrancava, processava os ficheiros e carregava a base de dados. Os pedidos foram organizados numa coleção do Postman (*ETL Pipeline – ISI*). A Figura 7 mostra os quatro pedidos e as respetivas respostas, e a Tabela 4 resume os cenários testados.

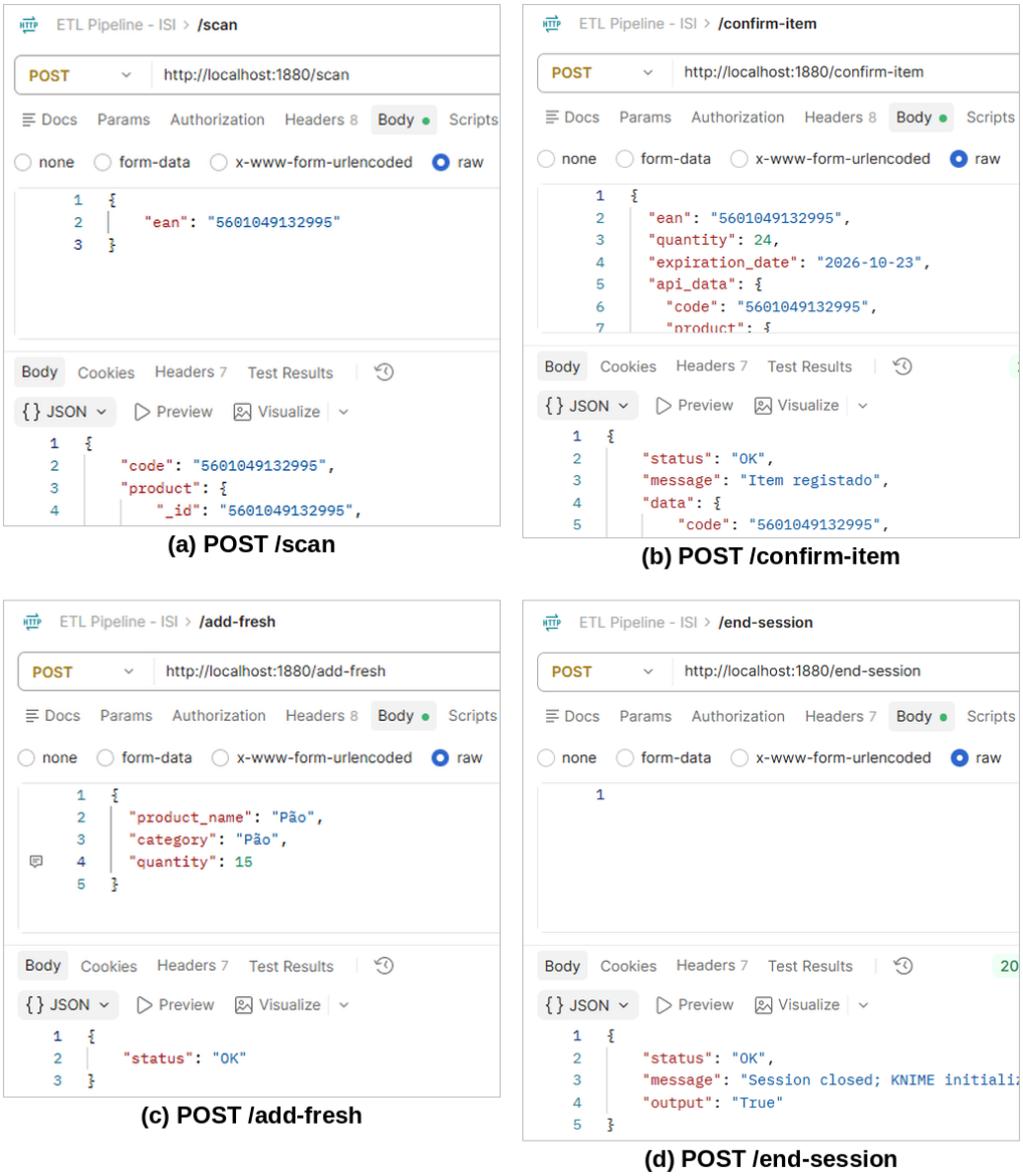


Figura 7: Testes aos endpoints do Node-RED no Postman.

Endpoint	Cenário de teste	Verificação
`/scan`	Envio de `{"ean": ...}` com um EAN existente e com um EAN inexistente na API.	Resposta JSON com `code` e `product`; ausência de resultado tratada sem falha do fluxo.
`/confirm-item`	Envio de `ean`, `quantity`, `expiration_date` e `api_data` para vários itens.	Linhas acrescentadas ao CSV e dados gravados no JSON; resposta JSON com `status`, `message` e `data`.
`/add-fresh`	Registo sucessivo de produtos frescos com `product_name`, `category` e `quantity`.	Acumulação dos itens em memória e resposta `{"status": "OK"}`.
`/end-session`	Pedido sem corpo, após uma contagem completa.	XML bem formado gravado em disco; resposta JSON com `status`, `message` e `output`.

Endpoint	Cenário de teste	Verificação
Node-RED → KNIME	Sessão completa terminada em `/end-session`.	Arranque do KNIME pelo nó exec, execução integral do workflow, dados nas tabelas do PostgreSQL e nova execução visível no Grafana.

Tabela 4: Cenários de teste executados com o Postman.

Os testes de integração revelaram-se particularmente úteis para detetar problemas que não são visíveis ao nível de cada componente, como o desalinhamento de esquemas na tabela `error_logs` e a necessidade de impor a ordem de escrita entre `products` e `inventory`. Por outro lado, mostraram também a principal limitação da integração por interface gráfica: como o Node-RED não recebe qualquer resultado da execução do KNIME, a verificação do sucesso do processamento teve de ser feita por inspeção direta da base de dados, dos *logs* e, mais tarde, do painel de execuções do Grafana e email. Os testes evidenciaram ainda alguma heterogeneidade nas respostas: inicialmente, `/scan` e `/add-fresh` devolviam JSON, enquanto `/confirm-item` e `/end-session` devolviam texto (no último caso, apenas `True`). O problema foi corrigido com os nós *String to JSON Object* descritos na secção 4.1, e os quatro *endpoints* passaram a responder em JSON, como se vê na Figura 7.

5.2. Síntese crítica das limitações

A Tabela 5 reúne as principais dificuldades encontradas, a solução efetivamente implementada e aquela que seria a abordagem ideal. Esta distinção é deliberada: as soluções adotadas resolveram os problemas com pragmatismo e cumprem os requisitos, mas algumas introduzem fragilidades ou dívida técnica que deve ser reconhecida.

Problema	Solução implementada	Abordagem ideal
Erros na execução do KNIME em modo batch.	Abertura do KNIME por GUI e simulação de atalhos de teclado.	Resolver o modo batch ou usar um servidor KNIME com interface REST e código de retorno.
`require('fs')` bloqueado nos nós function do Node-RED.	CSV sem cabeçalho; nomes atribuídos no KNIME com `Column Renamer`.	Disponibilizar o módulo ao nó por configuração, ou escrever o cabeçalho na abertura da sessão.
`DB Merger` substitui em vez de somar.	`DB Insert` em `temp_inventory` seguido de `DB SQL Executor` com lógica de conflito.	Solução adequada; acrescentar limpeza da tabela intermédia e execução transacional.

Problema	Solução implementada	Abordagem ideal
Produtos frescos sem validade e `expiration_date NOT NULL`.	Data convencional 2099-12-31 via `Missing Value`.	Chave primária independente da validade, permitindo valores nulos.
Regex e traduções no mesmo dicionário Excel.	Separação em dois passos: `Rule Engine` com regex devolve códigos; `Rule Engine (Dictionary)` só traduz códigos em nomes.	Solução adequada (implementada durante o projeto).
Desalinhamento de colunas em `error_logs`.	Validação manual do output do KNIME contra o esquema.	Validação automática de esquemas antes do *load*.
Categorias por normalizar, visíveis no Grafana.	Lista fechada de 17 categorias canónicas, com `OUTROS` por omissão.	Rever periodicamente os registos classificados como Outros e acrescentar palavras-chave.
Respostas HTTP heterogéneas (JSON, texto, `True`).	Nós `function` *String to JSON Object* nos fluxos `/confirm-item` e `/end-session`.	Envolver também `/scan` e `/add-fresh` num formato comum e usar códigos HTTP de erro.
Valores do campo *size* em formato livre, com texto que não interessa misturado com a medida.	Expressões regulares em nodes nativos do KNIME para isolar o valor e a unidade.	Solução adequada.
Medidas em unidades diferentes para a mesma grandeza.	*Script* Python integrado no *workflow*, com regex para detetar a unidade e aplicar a conversão.	Solução adequada.

Tabela 5: Síntese das dificuldades, soluções implementadas e abordagem ideal.

6. Conclusão

O projeto resultou num *pipeline* ETL funcional que transforma uma sessão de contagem de inventário alimentar num carregamento consistente de uma base de dados relacional. A separação de responsabilidades entre o Node-RED, focado na interação e na recolha, e o KNIME, focado no processamento e no *load*, revelou-se acertada: cada ferramenta foi usada naquilo em que é mais forte, e a fronteira entre ambas — ficheiros intermédios e um sinal de arranque — é simples de compreender e de testar.

Os requisitos de robustez, integridade e rastreabilidade foram cumpridos. As falhas pontuais são desviadas para a tabela `error_logs` com indicação da fase e do motivo, a ordem de escrita respeita as chaves estrangeiras e as quantidades são acumuladas corretamente graças ao padrão de tabela intermédia. O *dashboard* no Grafana fecha o ciclo, tornando visíveis os produtos próximos do fim da validade, a composição do stock e a qualidade de cada execução. Os testes com o Postman, tanto ao nível de cada *endpoint* como da integração entre o Node-RED e o KNIME, permitiram validar o comportamento de ponta a ponta e detetar problemas que só se manifestavam na interação entre componentes.

O trabalho evidenciou também a diferença entre resolver um problema e resolvê-lo da melhor forma. Várias limitações foram contornadas com soluções pragmáticas, e o relatório procurou documentar, para cada uma, qual seria a abordagem ideal. Duas delas, a normalização de texto e a heterogeneidade das respostas HTTP, foram resolvidas ainda durante o projeto, depois de identificadas no dashboard e nos testes. Das restantes, duas destacam-se pelo seu impacto: a execução do KNIME por interface gráfica, que é a principal fragilidade operacional do sistema, e a utilização de uma data de validade convencional, que é a principal fragilidade do modelo de dados.

6.1. Trabalho futuro

Como linhas de evolução, propõe-se:

- substituir a integração por interface gráfica pela execução em modo *batch* ou por um servidor KNIME com interface REST, de modo a que o Node-RED receba o resultado da execução;
- rever o modelo de dados, introduzindo uma chave primária independente da data de validade e eliminando a data convencional 2099-12-31;
- guardar o EAN em `products` com restrição de unicidade, ligar `error_logs` a `pipeline_executions` por chave estrangeira e uniformizar o tipo das quantidades (hoje `integer`, `numeric` e `double precision` consoante a tabela);
- configurar alertas no Grafana para produtos a poucos dias do fim da validade;
- tornar persistente o contexto do Node-RED onde se acumulam os produtos frescos, evitando perdas em caso de reinicialização;
- automatizar a validação de esquemas entre o *output* do KNIME e a base de dados, e envolver o *load* numa transação explícita;
- transformar os pedidos do Postman numa coleção com asserções automáticas, executável como teste de regressão.

Bibliografia

Anthropic. (2026). *Claude* (versão Opus 5.5) [Modelo de linguagem de grande escala]. Obtido de https://claude.ai

Grafana Labs. (s.d.). *Grafana Documentation*. Obtido de https://grafana.com/docs/grafana/latest/

KNIME AG. (s.d.). *KNIME Documentation*. Obtido de https://docs.knime.com/

OpenJS Foundation. (s.d.). *Node-RED Documentation*. Obtido de https://nodered.org/docs/

The PostgreSQL Global Development Group. (s.d.). *PostgreSQL Documentation*. Obtido de https://www.postgresql.org/docs/

Postman, Inc. (s.d.). *Postman Docs*. Obtido de https://learning.postman.com/docs/